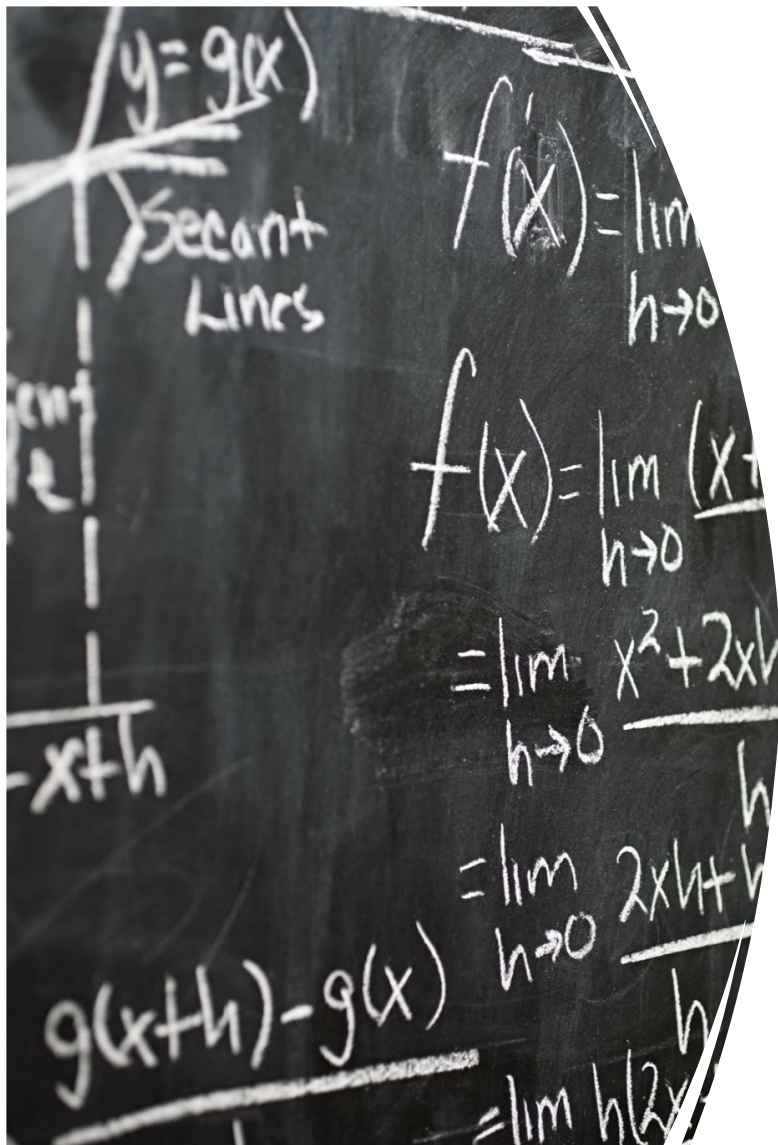




Lecture 13 -- Adversarial Attack on ML models

Prof Xiaowei Huang

<https://cgi.csc.liv.ac.uk/~xiaowei/>

(Attendance Code: **573451**)

Attendance Code: 573451

Topics

1

Attack on
Decision Tree

2

Attack on K-nn

3

Attack on
Logistic
Regression

will skip

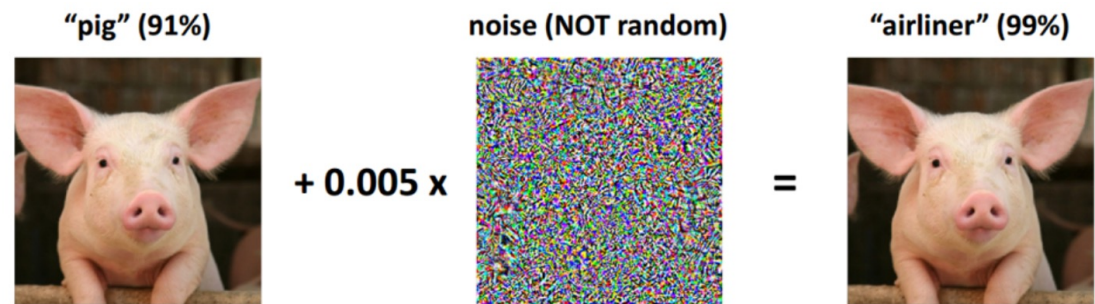
4

Attack on
Deep Learning

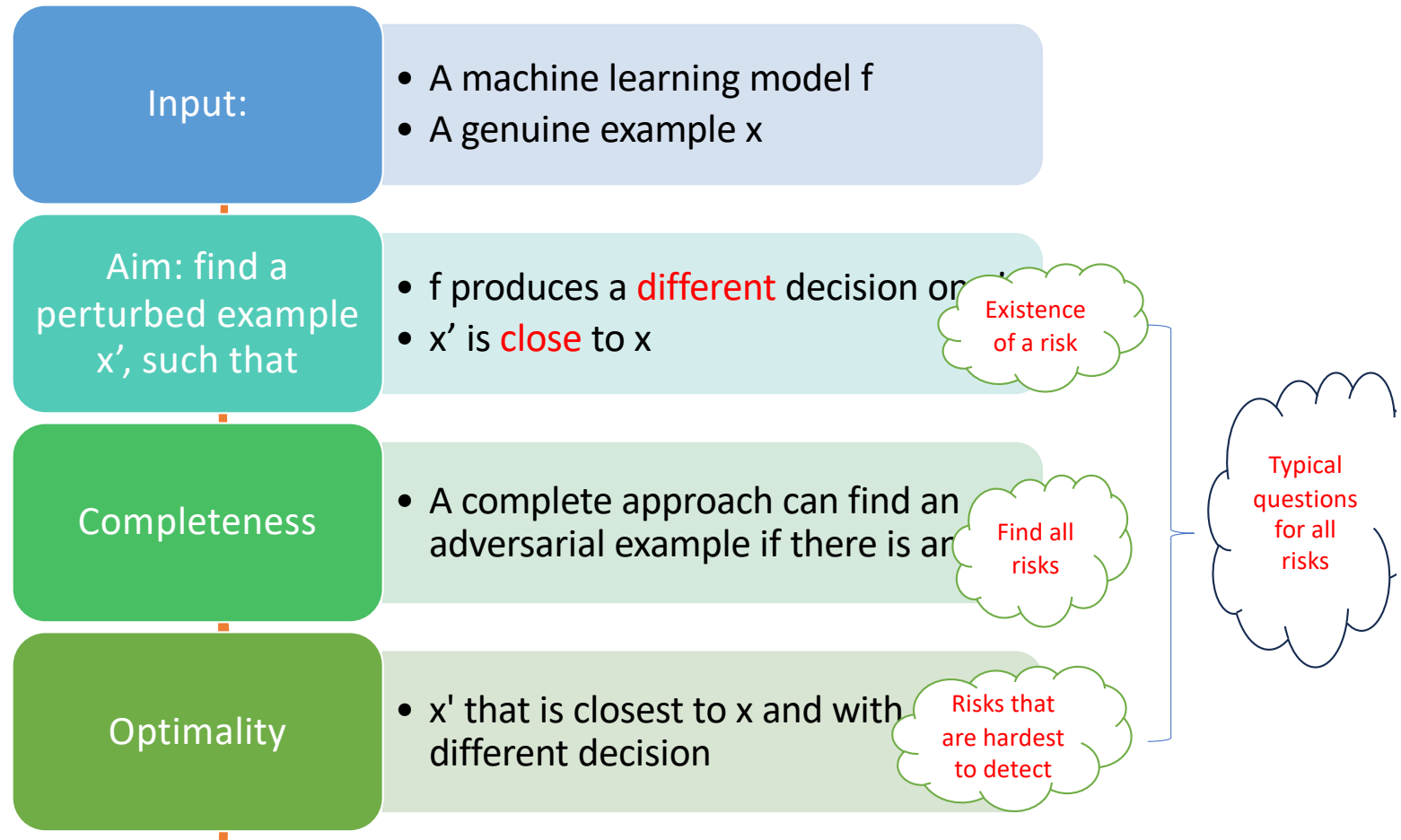
5

Defence on
Deep Learning

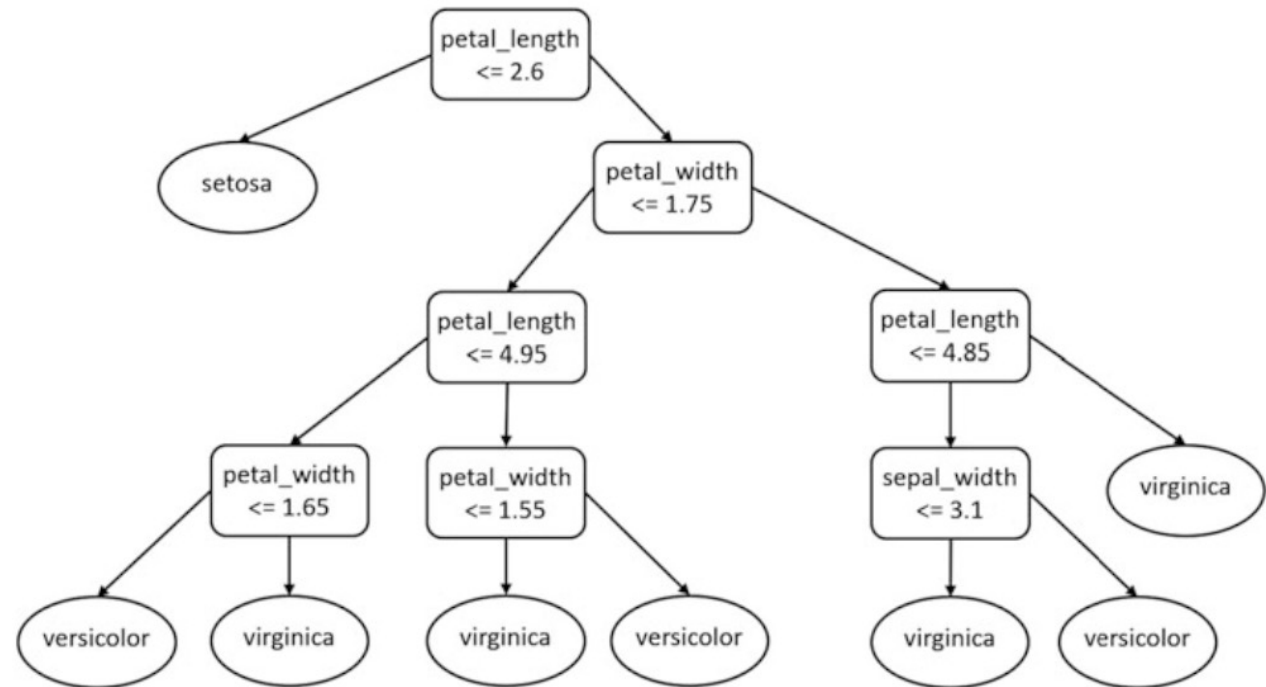
Illustration of Adversarial Attack



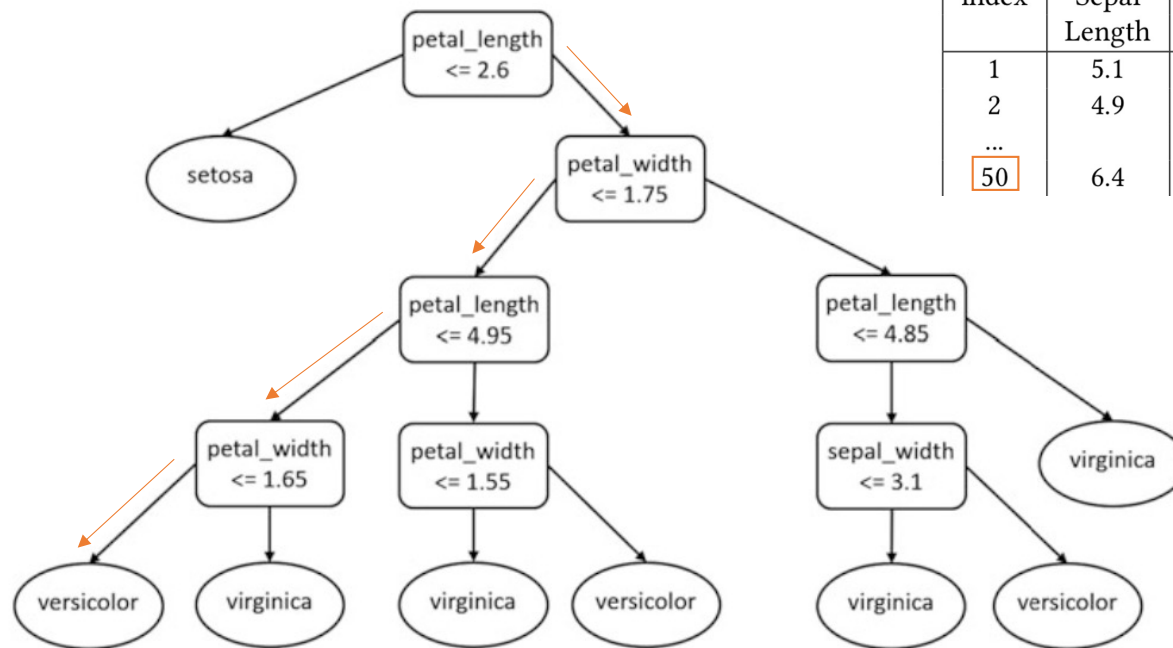
What is Adversarial Example



Adversarial Example for Decision Tree



Example



index	Sepal Length	Sepal Width	Petal Length	Petal Width	Class Label
1	5.1	3.5	1.4	0.2	iris setosa
2	4.9	3.0	1.4	0.2	iris setosa
...					
50	6.4	3.5	4.5	1.2	iris versicolor

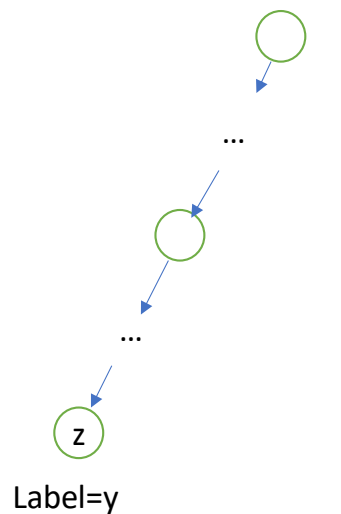
Adversarial attack for decision tree

Given an instance $\mathbf{x}$ and a decision tree:

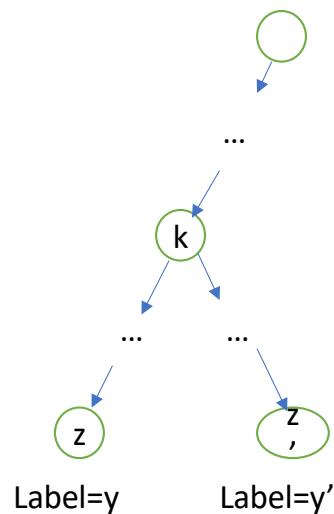
The algorithm proceeds with the following steps:

1. Given an input $\mathbf{x}$, it will lead to some leaf node z with label y .
2. Consider a targeted label $y' \neq y$, we find the shortest path from z to any leaf node with label y' . Let the new leaf node be z' .
3. Then, we can identify the common ancestor of z and z' on the shortest path, and construct a path from the root node to the common ancestor and then to z' .
4. Construct an input $\mathbf{x}'$ from the constructed path such that $\|\mathbf{x} - \mathbf{x}'\|$ is minimised. If $\|\mathbf{x} - \mathbf{x}'\| < \delta$ then we return $\mathbf{x}'$ as an adversarial example.

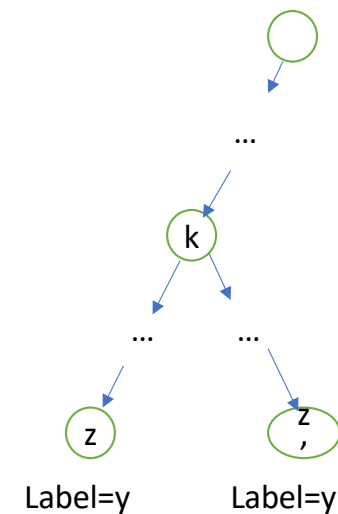
Illustration of Adversarial Attack Algorithm



(Step 1): get leaf node z with label y for input x

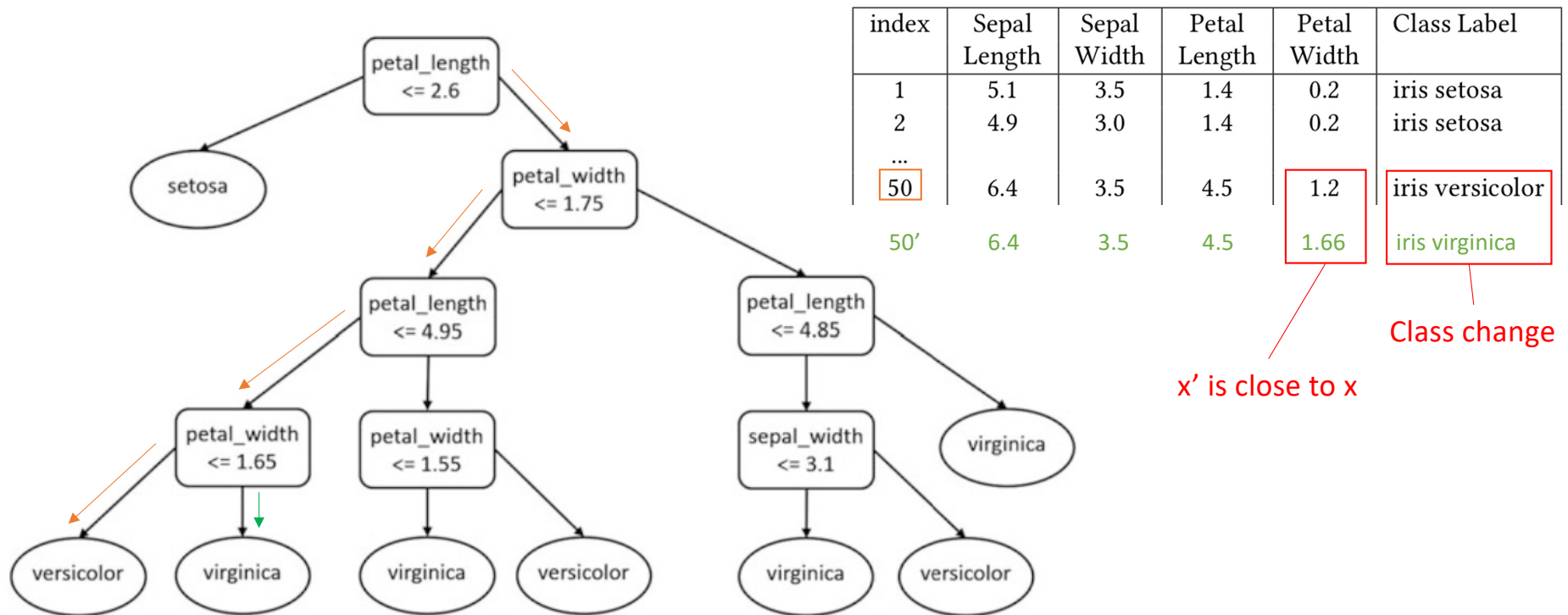


(Steps 2-3): find shortest path from z to any node with label y' . Let k be the common ancestor



(Step 4) Get adversarial example from root to z' through k

Example



Make petal_width=1.66

Attendance Code: 573451

Adversarial attack for decision tree

Is $\mathbf{x}'$ an adversarial example?

- Yes, because $\mathbf{x}'$ follows a path from the root node to the leaf node $\mathbf{z}'$, which is labelled as y' , different from y .

Existence
of a risk

Is this approach complete?

- A complete approach is able to find an adversarial example if there is any. Unfortunately, the above algorithm is incomplete.

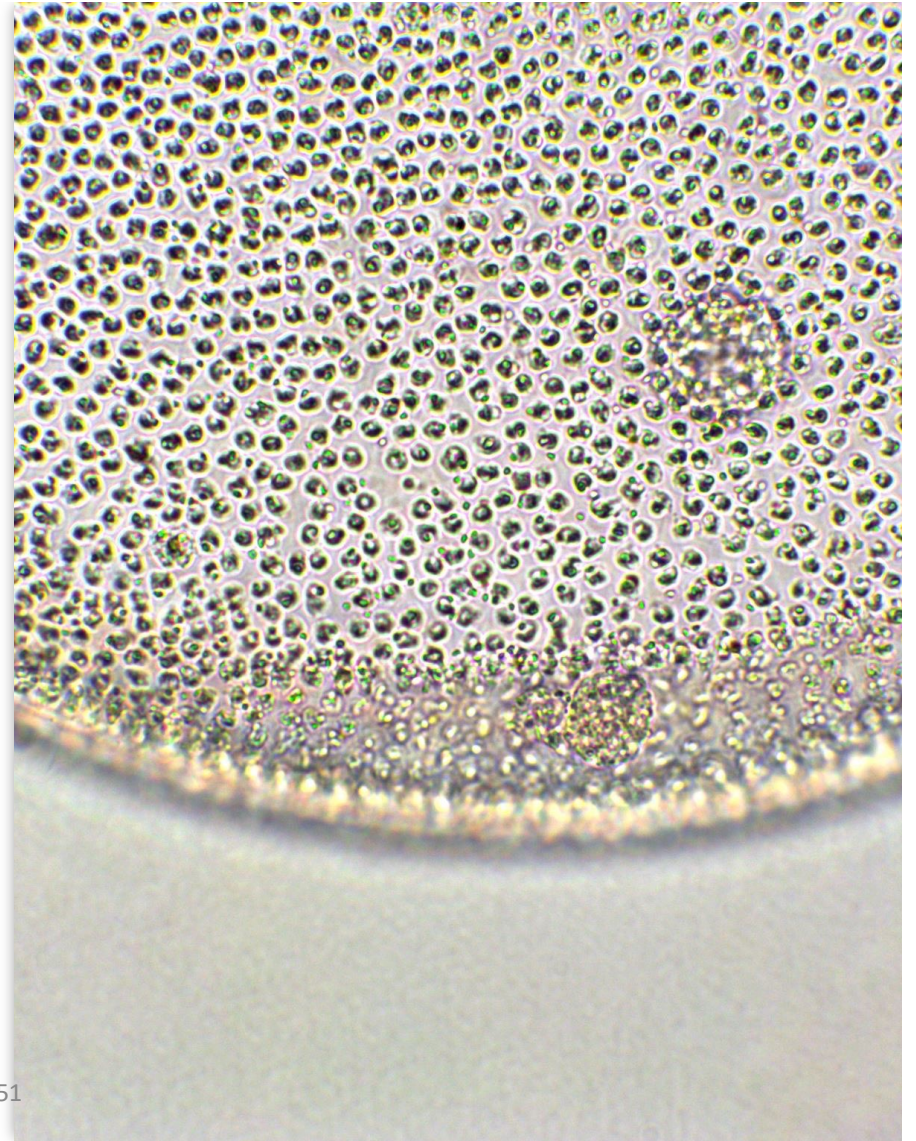
Find all
risks

Sub-optimality

- This algorithm is also sub-optimal, i.e., the found $\mathbf{x}'$ does not necessarily be the optimal solution to the optimisation problem.

Risks that
are hardest
to detect

Adversarial Example for K-nn

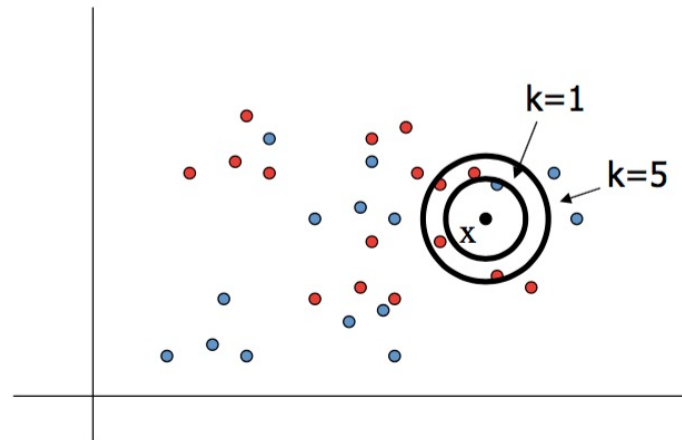


Attendance Code: 573451

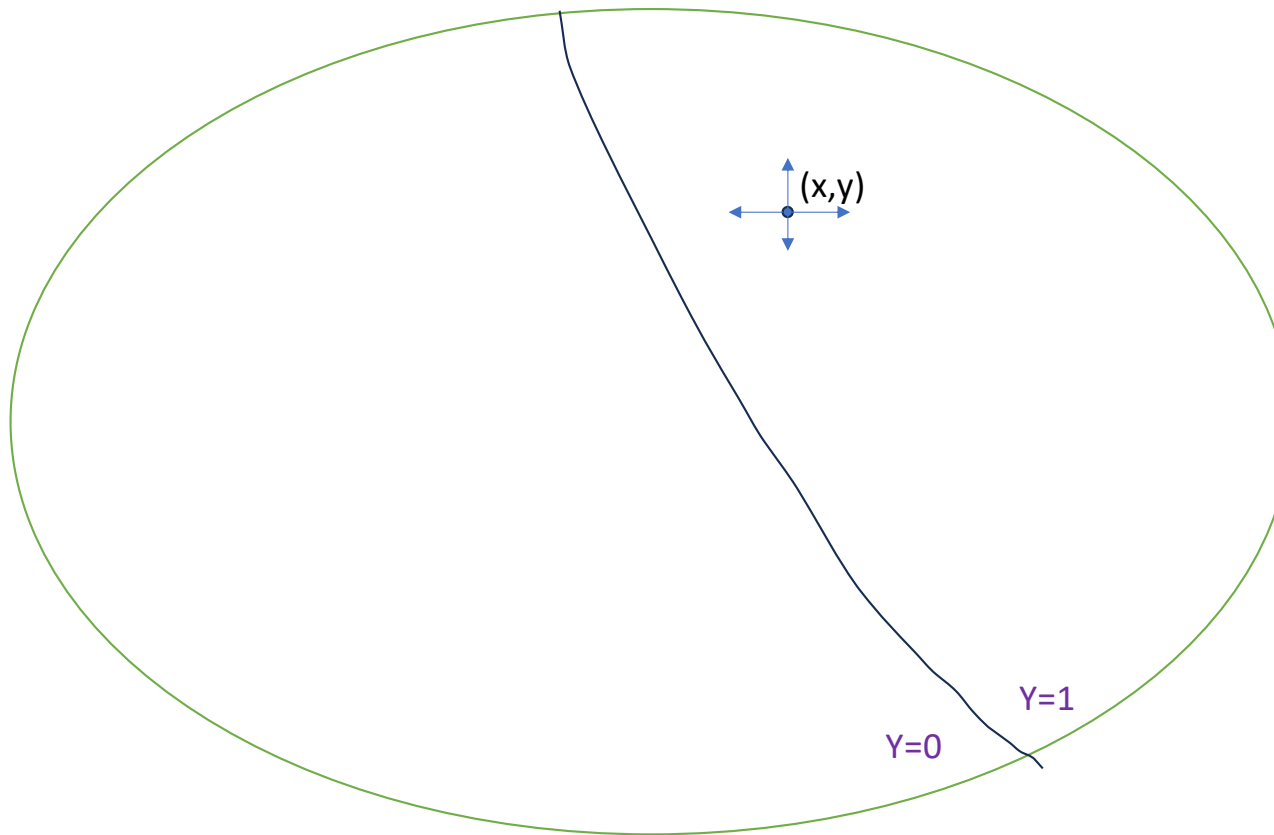
Adversarial attack for K-nn

To classify a new input vector x , examine the k -closest training data points to x and assign the object to the most frequently occurring class

- Assume that we have a data instance with label, (x, y) , and want to find another one (x', y') that is close to x .
- The general idea is to **move the instance x gradually along the direction** where the probability of being classified as y decreases the most, until the class change.



Example



Attendance Code: 573451

along the direction where the probability of being classified as y decreases the most, until the class change

Define

$$\nabla_i^s f(\mathbf{x}) = \frac{f(\mathbf{x}_i^s) - f(\mathbf{x})}{\|\mathbf{x} - \mathbf{x}_i^s\|}$$

Step 1

for all i and s , where i is the dimension and s is the direction

Adversarial attack for K-nn

First of all, we define

$$\mathbf{x}_i^s = \begin{cases} \mathbf{x} + \epsilon_i & \text{if } s=+ \\ \mathbf{x} - \epsilon_i & \text{if } s=- \end{cases} \quad (3.19)$$

where ϵ_i is a 0-vector except for the entry for feature X_i , which has value $\epsilon > 0$. Then, the gradient along the direction defined by X_i and $s \in \{+, -\}$ is as follows:

$$\nabla_i^s f(\mathbf{x}) = \frac{f(\mathbf{x}_i^s) - f(\mathbf{x})}{\|\mathbf{x} - \mathbf{x}_i^s\|} \quad (3.20)$$

Then, for the next step, we move $\mathbf{x}$ along the following direction to $\mathbf{x}'$:

Step 2

$$\arg \min_{i,s} \nabla_i^s f(\mathbf{x}) \quad (3.21)$$

and check if there is a misclassification on $\mathbf{x}'$. We repeat the above move until there is a misclassification.

the direction that decreases the most

Adversarial attack for K-nn

Is $\mathbf{x}'$ an adversarial example?

- Yes, because the final $\mathbf{x}'$ is obtained by following a sequence of changes until witnessing a misclassification.

Existence
of a risk

Is this approach complete?

- Unfortunately, the above algorithm is incomplete.

Find all
risks

Attendance Code: 573451

Sub-optimality

- The resulting $\mathbf{x}'$ does not necessarily be the optimal solution to the optimisation problem.

Risks that
are hardest
to detect

Adversarial Example for Logistic Regression



Attendance Code: 573451

Adversarial Example for Logistic Regression

- Considering a binary classification problem, i.e., $y \in \{+1, -1\}$ for any data instance $\mathbf{x}$, the loss for a single instance $(\mathbf{x}, y)$ is

$$\hat{L}(f_{\mathbf{w}}, (\mathbf{x}, y)) = -y \log \sigma(\mathbf{w}^T \mathbf{x}) - (1 - y) \log[1 - \sigma(\mathbf{w}^T \mathbf{x})]$$

- The computation of adversarial example is to solve the following optimisation problem:

$$\text{maximise}_{\|\delta\| \leq \epsilon} \quad \hat{L}(f_{\mathbf{w}}, (\mathbf{x} + \delta, y))$$

which finds the greatest loss within the constraint over the perturbation δ

Adversarial Example for Logistic Regression

$$\hat{L}(f_{\mathbf{w}}, (\mathbf{x}, y)) = -y \log \sigma(\mathbf{w}^T \mathbf{x}) - (1 - y) \log[1 - \sigma(\mathbf{w}^T \mathbf{x})]$$

- Note that, this is equivalent to

$$\text{minimise}_{\|\delta\| \leq \epsilon} y \mathbf{w}^T \delta$$

Step 1

- Now, considering the L_{∞} norm, i.e., $\|\delta\|_{\infty} \leq \epsilon$, the optimal solution is

$$\delta^* = -y \epsilon \text{sign}(\mathbf{w})$$

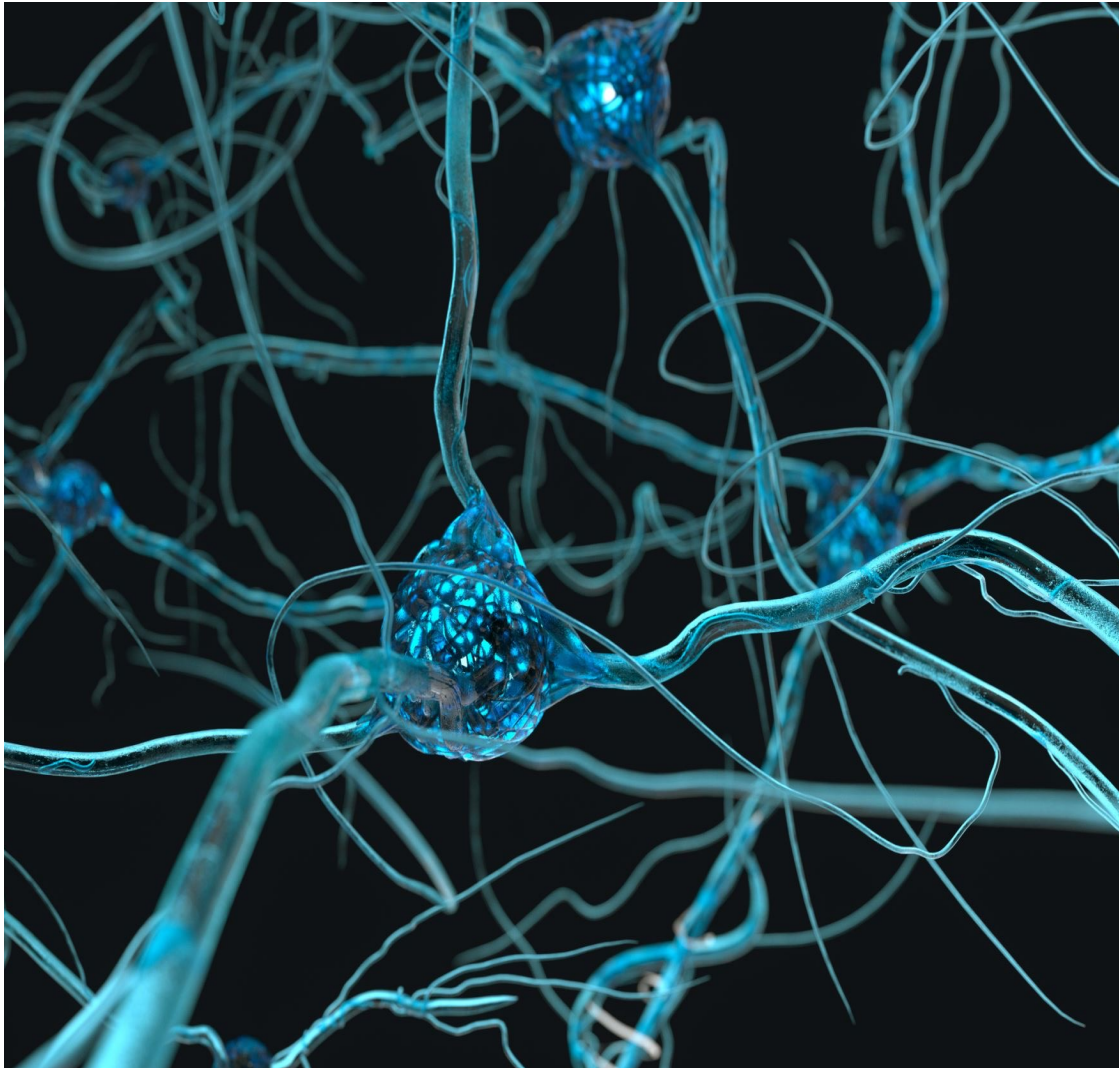
All components are known! So, we can compute analytically.

Step 3

where *sign* is the sign function extracting the sign of real numbers, such that

$$\text{sign}(w) = \begin{cases} -1 & w < 0 \\ 0 & w = 0 \\ 1 & w > 0 \end{cases}$$

Attendance Code: 573451

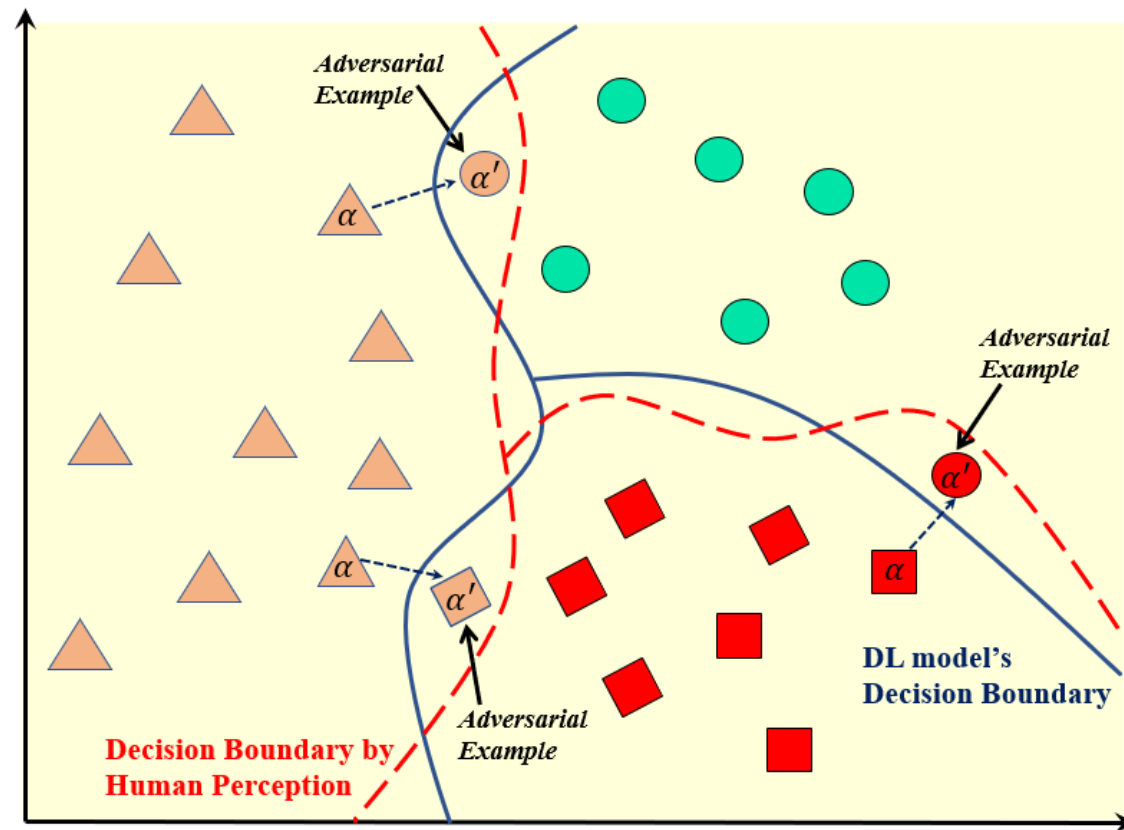


Adversarial Example for Convolutional Neural Networks

Illustration

- DL model: classifies α and α' **differently**
- Human: should remain the **same**

We have been ignoring the definition of “closeness” by now.



Definition

One of earliest adversarial attack: optimization based formulation with L_2 -norm metric

- ▶ Model $f : \mathbb{R}^{s_1} \rightarrow \{1 \dots s_K\}$ with s_K labels
- ▶ $x \in \mathbb{R}^{s_1} = [0, 1]^{s_1}$ is an input
- ▶ $t \in \{1 \dots s_K\}$ is a target misclassification label

Find the adversarial perturbation r via

$$\begin{aligned} \min & \|r\|_2 && \text{assure } \boxed{\text{human-decision}} \text{ unchanged} \\ \text{s.t.} & \arg \max_l f_l(x + r) = t && \text{assure misclassification} \\ & x + r \in \mathbb{R}^{s_1} && \text{assure perturbed image feasible} \end{aligned} \tag{1}$$

So, the "closeness" of two instances x and x' is defined over the human's ability (e.g., perception on images)

Expressed as constraints

Practically, we use L_0 , L_1 , L_2 , L_{inf} norms, but others are also used.

FGSM Attack

Fast Gradient Sign Method is able to find adversarial perturbations with a fixed L_∞ -norm constraint **very efficiently**

- ▶ θ : the model parameters,
- ▶ x, y : the input and the label
- ▶ $J(\theta, x, y)$: the loss function

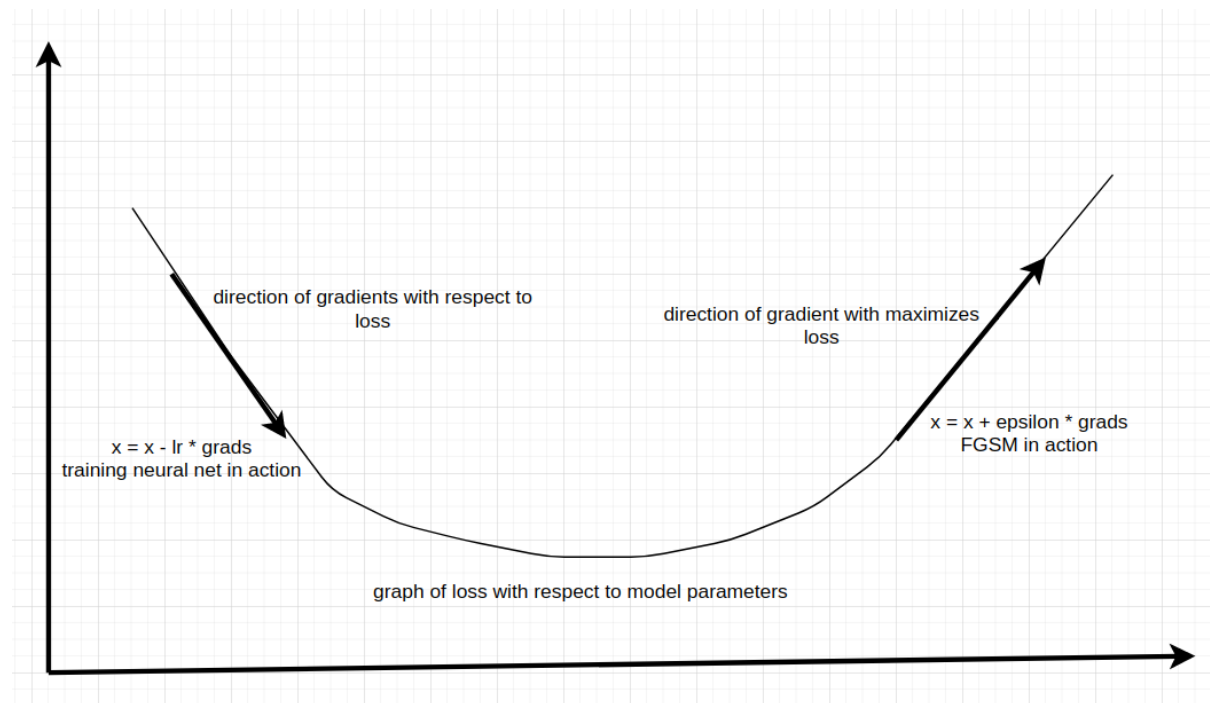
Find adversarial perturbation r by linearizing the loss function around the current value of θ ,

$$r = \epsilon \operatorname{sign}(\nabla_x J(\theta, x, y)) \quad (2)$$

- A **one-step modification** to all pixel values to increase the loss function with a L_∞ -norm constraint ϵ

Move along the the input gradient sign direction

Gradient Descent vs FGSM



Attendance Code: 573451

PyTorch Code for FGSM

https://pytorch.org/tutorials/beginner/fgsm_tutorial.html

```
# FGSM attack code
def fgsm_attack(image, epsilon, data_grad):
    # Collect the element-wise sign of the data gradient
    sign_data_grad = data_grad.sign()
    # Create the perturbed image by adjusting each pixel of the input image
    perturbed_image = image + epsilon*sign_data_grad
    # Adding clipping to maintain [0,1] range
    perturbed_image = torch.clamp(perturbed_image, 0, 1)
    # Return the perturbed image
    return perturbed_image
```

PyTorch has library function to compute gradient!

FGSM vs PGD (Projected Gradient Descent)

Aspect	FGSM (Fast Gradient Sign Method)	PGD (Projected Gradient Descent)
Type	<i>Single-step</i> attack	<i>Multi-step iterative</i> attack
Computation	One gradient computation → one update	Repeated gradient updates (e.g. 40 steps)
Formula	$x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x L(x, y))$	Iteratively: $x_{t+1} = \Pi_{x, \epsilon}(x_t + \alpha \cdot \text{sign}(\nabla_x L(x_t, y)))$
Projection step	 No projection	 Projects back into the allowed ϵ -ball after each step
Speed	Very fast	Slower (many iterations)
Strength	Weaker (easier to defend)	Stronger, often considered the "universal" first-order adversary
Purpose	Quick attack or baseline test	Robust adversarial training / strongest test

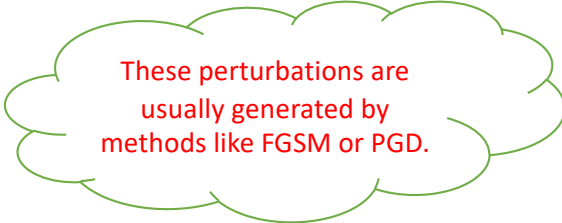
How to Reduce the Risks of Adv. Examples

- **Adversarial training**: train on both clean and adversarial data
- Regularisation and Smoothing: weight decay, dropout, etc
- Certified robustness
- Runtime detection
- Input pre-processing

Adversarial Training

- Adversarial training strengthens a model by exposing it to *adversarial examples* during learning.
- Instead of training only on clean data, the model is trained on inputs that have been deliberately perturbed to fool it.
- The formulation is

$$\min_{\theta} \mathbb{E}_{(x,y)} \left[\max_{\delta \in S} L(f_{\theta}(x + \delta), y) \right]$$



These perturbations are usually generated by methods like FGSM or PGD.

where the **inner maximization** creates the worst-case (adversarial) input within a small allowed perturbation set S , and the **outer minimization** trains the model parameters θ to perform well even under those attacks.

Illustrative diagram for adversarial training (from TRADES paper)

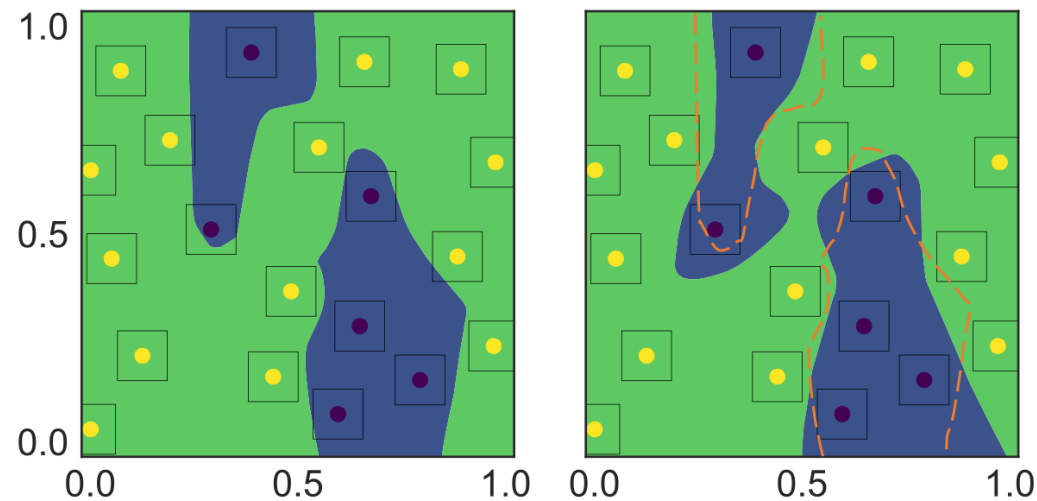


Figure 1: **Left figure:** decision boundary learned by natural training method. **Right figure:** decision boundary learned by our adversarial training method, where the orange dotted line represents the decision boundary in the left figure. It shows that both methods achieve zero natural training error, while our adversarial training method achieves better robust training error than the natural training method.

More information

- If you are interested in this topic (and other related topics), you can check

[Lecture 14 \(Supplementary\) -- IJCAI2021 tutorial](#)

in Canvas